

PROVISIONAL PATENT APPLICATION

Title: Replayable Write-Side Receipts and Counterfactual Policy Evaluation for Motive-Governed Autonomous Agent Memory

Filing type: U.S. Provisional Application under 35 U.S.C. § 111(b).

FIELD OF THE INVENTION

[0001] The present disclosure relates to computer systems for autonomous agent memory. More particularly, it relates to policy-governed formation of durable memory from conversations, documents, tool results, and multimodal artifacts; to canonical, tamper-evident receipts that record each write-side memory decision; to deterministic replay and verification of a memory-store state from those receipts and immutable evidence; to counterfactual evaluation of an alternate memory-formation policy against the same evidence without mutating production memory; and to bounded retrieval from a context-visible working memory tier while preserving a full evidence tier.

BACKGROUND

[0002] Autonomous agents increasingly extract facts, preferences, requirements, directives, decisions, and other durable context from streams of evidence and store the result in a long-lived memory. Such systems commonly provide a memory store, periodic background consolidation of stored memories, semantic search, summarization, temporal or graph storage, per-tenant configuration, and generic diagnostic logging. A non-deterministic language model typically performs the extraction, so the contents of memory are an opaque function of prior model calls.

[0003] These features do not solve the failure modes that govern whether such a memory can be operated in a regulated or high-assurance enterprise. First, an operator cannot prove why an agent committed one extracted candidate to memory but rejected another. Second, an operator cannot reconstruct, from the recorded evidence alone, the memory state that should have existed at a prior point in time. Third, an operator cannot safely ask what the memory state would have been under a stricter or different policy without mutating production memory. Fourth, a compliance reviewer cannot distinguish a fact that was never observed from a fact that was observed but deliberately not persisted. Fifth, a tenant cannot certify that a named memory policy avoids persisting sensitive or disallowed memory types across a benchmark corpus. Sixth, a memory audit log can itself be incomplete, mutable, or disconnected from the memory rows it allegedly explains.

[0004] Existing approaches do not solve these problems together. Background memory-consolidation systems reorganize or summarize stored memories on a schedule, but neither record candidate-level write-side decisions nor replay the resulting memory state. Agent memory stores persist long-term memories and retrieve them for an agent, but do not record why a candidate was or was not materialized, and cannot reconstruct that state from a decision record. Temporal knowledge graphs store time-varying facts, but do not bind a graph mutation to a named policy version, a candidate record, and a tamper-evident commitment. Generic audit logs record diagnostic events emitted after a decision, but are not canonical, not candidate-bound, not policy-bound, and not replayable into a memory state. Generic hash-chain or Merkle-tree ledgers provide tamper evidence over arbitrary records, but do not bind each linked entry to the semantics of an autonomous memory-formation decision. There is therefore a need for the systems and methods described herein.

SUMMARY

[0005] Disclosed are systems, methods, and computer-readable media that turn the write-side memory-formation pipeline of an autonomous agent into an executable receipt stream. As each candidate memory is extracted from an evidence episode and admitted, transformed, reinforced, superseded, demoted, gated, or rejected under a named memory-formation policy object, the system emits a canonical, schema-versioned receipt that binds the decision to a digest of the immutable evidence, a digest of the resolved policy, a digest of the candidate, the decision reason, and the resulting memory-store pointers. The receipts are canonicalized to a stable byte representation, hash-linked into a tamper-evident sequence, and committed into a per-run commitment, so that a memory-store state can be replayed, verified, and counterfactually regenerated under a different policy without silently changing raw evidence.

[0006] A named memory-formation policy object, referred to herein as a Motive, specifies at least an allowed memory-type set and one or more write-side memory controls, and in embodiments further specifies a salience rubric, a prompt profile, a semantic-deduplication threshold, a retrieval-budget share, and a governance policy. A catalog of such policy objects, referred to herein as a MemoryBank, is available to a tenant, agent, job, or scope. The system resolves an effective policy for an episode from a layered precedence of tenant, agent, job, scope, dream mode, and request override, and computes a policy version digest that binds the resolved policy object to all inherited inputs, so that the exact policy under which a memory was formed is itself a verifiable artifact.

[0007] In embodiments, the system comprises a receipt recorder that captures decision events during formation, consolidation, pruning, redaction, and retrieval-budget rendering; a canonicalizer that converts each decision payload into a stable byte representation; a receipt hash chain that links each canonical receipt to a preceding receipt; a run checkpoint that commits the receipts of a run into a Merkle root; a replay engine that reconstructs a memory delta or full memory state by applying receipts to immutable evidence and versioned policy artifacts; a counterfactual evaluator that replays the same evidence and candidate stream under an alternate policy to produce a non-mutating delta; a negative-space query interface that returns candidates observed but not materialized together with the gate, score, threshold, policy, and evidence that explain each rejection; a certification harness that runs a policy object against a fixture corpus and emits a receipt-backed certificate; and a tuning analyzer that mines historical receipts to propose, but not activate, a revised policy.

[0008] The disclosed systems provide, in embodiments, a replayable memory state that is reconstructed or verified from receipts and immutable evidence rather than trusted as an opaque result of prior model calls; a fail-closed audit in which a missing policy version, a missing candidate receipt, a broken receipt chain, or a graph-state-hash mismatch prevents issuance of a replay proof; a bounded retrieval that reads a context-visible working tier through an index while the full evidence tier remains available for search and audit; a counterfactual policy evaluation that compares an alternate policy against the same evidence in isolation from production memory; a negative-space provenance that records candidates deliberately filtered, gated, redacted, reinforced, superseded, demoted, or pruned, not only memories that exist; a tamper evidence in which hash-chain and Merkle proofs bind memory-specific decision inputs and graph outputs rather than generic log text; and a certifiable memory policy that can be benchmarked and certified through receipt proofs. A receipt differs from a diagnostic log entry in that it is a canonical input/output contract sufficient to reconstruct memory state, rather than descriptive text emitted after a decision.

BRIEF DESCRIPTION OF THE DRAWINGS

[0009] FIG. 1 is a block diagram of the receipt-backed memory-formation architecture and its replay, counterfactual, negative-space, and certification consumers.

[0010] FIG. 2 is a diagram of the receipt record, the receipt hash chain, and the per-run Merkle checkpoint.

[0011] FIG. 3 is a flowchart of the receipt-backed memory-formation method.

[0012] FIG. 4 is a flowchart of the byte-replay method, including the fail-closed verification gate.

[0013] FIG. 5 is a dataflow diagram of counterfactual policy evaluation against a recorded evidence and candidate stream without mutating production memory.

[0014] FIG. 6 is a dataflow diagram of the negative-space memory query.

[0015] FIG. 7 is a flowchart of the policy certification harness over a fixture corpus.

DETAILED DESCRIPTION

Architecture overview

[0016] Referring to FIG. 1, an agent or application (100) supplies an evidence episode (105) associated with at least one of a tenant, a scope, and an agent. An evidence-digest module (110) computes an immutable digest of the episode body, its normalized source metadata, and its scope. A policy source comprising a control plane and a MemoryBank (115) provides a layered configuration from which a Motive resolver (120) resolves an effective memory-formation policy object and computes a policy version digest and an effective policy digest. A candidate extraction module (125), operating under a recorded prompt, model, and profile snapshot, produces one or more candidate memory records and records candidate receipts (130). A bank of policy gates (135) applies type filtering, salience thresholding, governance redaction, trust gating, semantic deduplication, and supersession to each candidate, recording decision receipts (140) for every transformation, block, reinforcement, supersession, creation, or rejection. A graph-mutation module (145) applies the determined disposition to a typed temporal property-graph memory store, and a graph-state-hash module (150) computes a digest of the store before and after each mutation. A receipt hash chain (155) links each canonical receipt to its predecessor, and a run checkpoint (160) commits the receipts of a run into a Merkle root. Downstream of the receipt store, a replay engine (165) reconstructs or verifies memory state; a counterfactual evaluator (170) applies an alternate policy to the recorded candidate stream; a negative-space query interface (175) returns observed-but-unpersisted candidates; a certification harness (180) certifies a policy over a fixture corpus; and a tuning analyzer (185) proposes revised policies.

[0017] Non-deterministic computation is confined to the extraction and optional advisory components; the policy gates, the canonicalizer, the hash chain, the checkpoint, and the replay engine are deterministic and auditable. The memory store comprises a context-visible working tier and a full evidence tier. The working tier is the default retrieval surface and is read through an index; the evidence tier retains superseded, pruned, redacted, and demoted relationships with their validity windows and evidence lineage and is available to search, evidence, and historical reads. Because every write-side decision is recorded as a receipt, and because the receipts are canonical and hash-linked, the memory state is causally reconstructable and the audit trail cannot silently diverge from the graph rows it explains.

Definitions

[0018] As used herein, an *episode* is a bounded unit of raw evidence, such as a chat window, a document chunk, a tool trace, a transcript span, or a normalized multimodal artifact. An *evidence digest* is a collision-resistant digest of the episode body, its normalized source metadata, and its scope. A *Motive* is a named memory-formation policy object that may specify allowed memory types, a salience rubric, a prompt profile, a deduplication threshold, a retrieval-budget share, a governance policy, and goal text. A *Motive version* is a canonical digest of the Motive together with all inherited policy inputs resolved for a run, including tenant, agent, scope, prompt pack, dream mode, governance, deduplication, and pruning policy. A *candidate memory* is a structured memory proposal extracted from an episode before final materialization, including subject, predicate, object, relationship type, memory type, confidence, salience, source span, and evidence digest. A *decision event* indicates how a candidate or graph relationship was treated, such as extracted, type-filtered, salience-filtered, governance-redacted, trust-gated, dedup-reinforced, superseded, created, demoted, pruned, or theme-gated. A *receipt* is a canonical, schema-versioned, hash-linked decision event carrying the inputs, policy identity, decision reason, and resulting graph pointers for a memory operation. *Negative space* is the set of candidates observed by the system but absent from active memory because they were rejected, gated, transformed, demoted, redacted, reinforced into an existing row, superseded, or pruned. *Byte replay* is the reconstruction of a memory delta from stored candidates, scores, policy digests, and decision receipts without invoking a language model. *Regenerative replay* is the re-running of extraction or theme synthesis from raw evidence under a specified model, prompt, and Motive snapshot, followed by comparison of the result to the stored receipt stream. *Counterfactual evaluation* is the application of a different Motive to the same episode and candidate stream to produce an isolated proposed memory state or delta without mutating production memory.

Receipt data model

[0019] Referring to FIG. 2, a receipt ledger row (200) of a memory-receipts table comprises, in embodiments, a receipt identifier, a schema version, a tenant identifier, an agent identifier, a scope key, a run identifier, an episode identifier, an episode digest (212), an event index, an event type, a decision type, a candidate identifier, a candidate digest (210), a source-span digest, a Motive name, a Motive version digest (214), an effective policy digest (216), a prompt profile and version, a model identifier, an extractor identifier, an embedding identifier, a memory type, a relationship type, a truth key, a salience score, a salience threshold, a deduplication threshold, a deduplication match identifier, a deduplication score, a governance policy digest (218), a redaction digest before and a redaction digest after transformation, a decision reason, a decision result (220), a graph relationship identifier, a graph state hash before (150) and a graph state hash after the mutation, a canonical payload, a previous receipt hash (230), a receipt hash (232), and a creation time. The row need not store raw sensitive text; it may store digests, redacted text, encrypted content, source pointers, and graph relationship identifiers. Sensitive payload fields may be protected by a destroyable per-scope content key while the receipt skeleton remains.

[0020] A candidate receipt records the structured output of extraction before any policy gate is applied, including digests or redacted values of the candidate subject, predicate, and object; the resolved relationship type and memory type; a confidence score; a salience score; the evidence episode and source span; a digest of the extractor prompt, profile, and model; and the candidate digest (210). Because the candidate digest is computed before policy gating, a later counterfactual evaluation can determine whether the same candidate would have been accepted or rejected under

a different policy.

[0021] Representative decision-type values recorded in receipts include `candidate_extracted`, `candidate_schema_rejected`, `candidate_low_salience`, `formation_motive_type_filtered`, `formation_governance_redacted`, `formation_untrusted_directive_gated`, `formation_semantic_dedup_reinforced`, `formation_truth_key_superseded`, `formation_relationship_created`, `consolidation_motive_theme_gated`, `consolidation_cluster_summarized`, `consolidation_member_demoted`, `pruning_soft_cap_pruned`, `redaction_relationship_version_redacted`, `crypto_shred_key_destroyed`, `retrieval_budget_overflow_referenced`, `counterfactual_candidate_accepted`, and `counterfactual_candidate_rejected`.

[0022] A run checkpoint (240) stored with each formation, consolidation, or pruning run comprises, in embodiments, a run identifier, a run kind, a Motive version digest, an effective policy digest, a first receipt hash, a last receipt hash, a receipt count, a Merkle root (250), a graph state hash before the run, a graph state hash after the run, a replay status, and a creation time. The Merkle root may be computed per run, per scope, per tenant, or globally, and may be exported to external tamper-evident storage; the disclosed system does not require a blockchain or any particular external ledger.

Canonicalization and hashing

[0023] Referring again to FIG. 2, a canonicalizer converts each receipt payload into a stable byte representation before hashing. In embodiments the canonicalizer normalizes text according to a configured normalization profile; redacts or encrypts sensitive fields before hashing where required; encodes timestamps in coordinated universal time with fixed precision; encodes floating values, including salience and deduplication scores, with fixed precision or a rational representation; sorts object keys and array entries where order is semantically irrelevant; and preserves explicit ordering where order is part of the computation, such as candidate ordering and the decision event index. The canonicalizer then computes a payload digest and a receipt hash as follows:

```
payload_digest = H(canonical_payload)
receipt_hash   = H(schema_version || previous_receipt_hash
                  || payload_digest || run_uuid || event_index)
```

For a per-run Merkle root, the leaf hashes are the receipt hashes ordered by event index and the resulting root is committed to the run record (240). A change to any receipt field, to the order of receipts, or to the membership of the run changes the receipt hash and the Merkle root, providing tamper evidence bound to the memory-formation decision rather than to generic log text.

Receipt-backed memory formation

[0024] Referring to FIG. 3, the receipt-backed formation method comprises: receiving an episode in a tenant and scope (300); computing an immutable evidence digest for the episode (305); resolving an effective Motive and policy from the tenant, agent, job, scope, dream mode, request override, and MemoryBank, and computing a Motive version digest and an effective policy digest (310); extracting candidate memories under a recorded prompt, model, and profile snapshot (315); recording a candidate receipt for each candidate (320); resolving each candidate's memory type and truth key (325); applying governance redaction or trust gates and recording a receipt for each transformation or block (330); applying Motive type gates and salience thresholds and recording a receipt for each accepted or rejected candidate (335); comparing each accepted candidate to active relationships by

truth key and semantic deduplication, and recording the match score and threshold (340); creating, reinforcing, superseding, or rejecting a graph relationship and recording a receipt for each mutation, including graph state hashes before and after the mutation (345); linking each receipt into the hash chain and computing a run checkpoint Merkle root (350); and marking the run replay-verifiable only if the final graph state hash matches the receipt-derived graph state hash (355). No candidate extracted from the episode is silently dropped: every candidate produces at least a candidate receipt, and every disposition produces a decision receipt.

Byte replay

[0025] Referring to FIG. 4, byte replay reconstructs a memory delta without invoking a language model. The method comprises: selecting a run, scope, time range, episode set, or receipt range (400); verifying the receipt chain and the run Merkle root (410); loading the immutable episode digests, candidate receipts, policy digests, and decision receipts (420); and, if a required raw evidence digest, candidate receipt, Motive version, policy digest, or graph predecessor row is missing (430), failing closed and withholding a replay proof (435). Otherwise, starting from a graph state hash or an empty state, the method applies the decision receipts in event order (440); recomputes relationship identifiers, truth keys, status transitions, demotion flags, supersession pointers, and graph state hashes (450); and compares the reconstructed graph state hash to the stored run checkpoint (460). When the reconstructed state matches, the method returns a replay proof (470); when it does not match, the method returns a mismatch report and fails closed (480). Byte replay is applicable to audits, regression tests, migration validation, incident response, and tenant export, and because it does not invoke a language model, one recorded run yields arbitrarily many identical reconstructions.

Counterfactual policy evaluation

[0026] Referring to FIG. 5, counterfactual evaluation determines what the memory state would have been under a different policy. The method comprises: selecting an episode set, an existing run, or a receipt range and loading the recorded candidate receipts and evidence (500); optionally regenerating candidate memories under a specified extraction snapshot; resolving an alternate Motive, Motive version, and effective policy (510); applying the alternate policy’s type gates, salience rubric, deduplication threshold, governance policy, retrieval-budget share, and theme-admissibility rules to the candidate stream (520); producing an isolated memory delta with accepted, rejected, transformed, reinforced, superseded, demoted, and pruned outcomes (530); comparing the alternate delta to the production delta (540); and emitting a counterfactual report and optional receipt stream (550) while leaving the production graph rows unchanged (560). In embodiments the report identifies candidates accepted by both policies, candidates accepted only by the original policy, candidates accepted only by the alternate policy, candidates rejected by both with reasons, sensitive candidates that would be persisted by one policy but not another, theme relationships that would or would not be synthesized, demoted members under each policy, retrieval-budget changes in the default profile, and a graph-state hash of the counterfactual memory view. Because the candidate digest is computed before any gate, the counterfactual delta is derived deterministically from recorded candidates without re-invoking the extractor, and the production memory store is never mutated.

Negative-space memory query

[0027] Referring to FIG. 6, the negative-space query answers what the system chose not to remember. The method comprises: receiving a query over scope, time, episode, Motive, memory type, or decision reason (600); selecting receipt rows whose decision result is non-materializing, such as filtered, gated, redacted, reinforced, superseded, demoted, or pruned (610); joining each receipt to its candidate digest, evidence pointer, policy digest, Motive digest, and decision reason (620); applying a redaction filter so that the explanation is policy-safe (630); and returning a redacted explanation suitable for an operator, an auditor, or a user (640). For example, an operator may request every customer-support episode in which a candidate directive was extracted but not persisted because the active Motive allowed only requirement memory; or a privacy reviewer may request every sensitive candidate considered in a scope together with whether it was redacted, hashed, rejected, or persisted in encrypted form. The negative-space query thereby converts the absence of a memory into an explained, evidence-backed record rather than an unexplained gap.

Policy certification

[0028] Referring to FIG. 7, a Motive may be certified over a benchmark corpus. The method comprises: selecting a fixture corpus with known sensitive, irrelevant, contradictory, low-salience, and high-value memory candidates (700); running the formation pipeline under the Motive (710); recording receipts for every candidate and decision (720); verifying the receipt chain and the graph state hash (730); computing certification metrics including a disallowed-type persistence count, a sensitive-data persistence count, an unexplained-candidate count, a duplicate-memory rate, a supersession-correctness count, a context-visible memory count, a demoted-relationship count, and a retrieval-budget allocation by memory type (740); and emitting a Motive certificate that includes the Motive digest, the corpus digest, the run Merkle root, the metrics, the failing receipts, and a pass/fail status (750). The certificate converts a Motive from a configuration object into a testable policy artifact whose behavior over a corpus is bound to a verifiable receipt root.

Receipt-derived tuning

[0029] In embodiments, the system mines historical receipts to propose Motive improvements. The method comprises aggregating decisions over many runs; identifying repeated friction patterns, such as many near-threshold rejected candidates that users later add manually; estimating alternative salience thresholds, allowed memory types, governance settings, or deduplication thresholds; running counterfactual evaluation of the proposed change before any activation; emitting a proposed Motive version with receipt-backed rationale; and requiring human, administrator, or tenant approval before the proposed Motive becomes active. The tuning analyzer is review-first and does not silently rewrite an active memory policy.

Worked example

[0030] In one example, a support agent processes a customer conversation under a Motive **learn-compliance-requirements** that allows requirement memory and applies a high-sensitivity governance policy. Extraction produces three candidates: a first candidate that the customer requires a SOC 2 report before contracting; a second candidate that the customer prefers meetings after 3 PM; and a third candidate that the customer provided a personal phone number.

[0031] The system records a candidate receipt for each candidate and then a decision receipt for each disposition. The first candidate is accepted and materialized because its memory type is

requirement, its salience exceeds the threshold, and governance permits the fact; the receipt binds the created relationship identifier and the graph state hashes before and after the mutation. The second candidate is type-filtered because preference is not in the Motive’s allowed memory-type set; the receipt records the type gate and a candidate digest but no graph mutation. The third candidate is detected as sensitive, redacted or gated by governance, and not persisted as active memory; the receipt records the redaction digests before and after and a non-materialization reason.

[0032] An auditor can later prove from the receipts that the phone number was observed but not persisted, that the preference was dropped because of Motive type filtering, and that the SOC 2 requirement created a graph relationship. Counterfactual evaluation under an alternate Motive `build-customer-profile` shows, without mutating production memory, that the preference candidate would have been accepted while the sensitive candidate remains gated by governance.

Generalizations

[0033] The memory store may be any typed temporal store, including a property graph, and the evidence may be of any modality, including text, code, structured data, images, and normalized multimodal artifacts. The hash and digest functions may be any collision-resistant functions, and the per-run commitment may be a Merkle root or any equivalent commitment. The named policy object need not be called a Motive, and the catalog need not be called a MemoryBank. The system may be embodied in a single-tenant or multi-tenant deployment, and the receipts may be stored in the same database as the memory store or in a separate tamper-evident store. Numerical examples and field names are illustrative and non-limiting.

CLAIMS

Invention I — Replayable receipt-backed memory formation

1. A computer-implemented method for forming and verifying autonomous agent memory, comprising: receiving an evidence episode associated with at least one of a tenant, a scope, and an agent, and computing an evidence digest of the episode; resolving a named memory-formation policy object from a catalog, the policy object specifying at least an allowed memory-type set and one or more write-side memory controls, and computing a policy version digest that binds the policy object to inherited policy inputs resolved for the episode; generating, from the episode, one or more candidate memory records, each candidate memory record having a candidate digest computed before application of the policy object; for each candidate memory record, applying the policy object to determine a disposition selected from materialize, transform, reinforce into an existing memory, supersede, demote, gate, and reject, and applying the determined disposition to a typed temporal memory store; storing, for each said determination, a canonical receipt comprising the evidence digest, the policy version digest, the candidate digest, a decision reason, and at least one of a resulting memory-store pointer and a non-materialization reason; linking the canonical receipts into a tamper-evident sequence by a hash that binds each receipt to a preceding receipt; and reconstructing or verifying a state of the memory store by replaying the canonical receipts against the evidence digests and the policy version digest.

2. The method of claim 1, wherein reconstructing or verifying the state of the memory store by replaying the canonical receipts is performed without invoking a language model, whereby one recorded run yields arbitrarily many identical reconstructions of the memory state.

3. The method of claim 1, further comprising committing the linked canonical receipts of a formation run into a Merkle root stored with a run record, and withholding a replay proof when a receipt hash, the policy version digest, an evidence digest, or a graph state hash does not match, whereby verification fails closed.

4. The method of claim 1, wherein each canonical receipt of a memory-store mutation further comprises a state hash of the memory store before the mutation and a state hash of the memory store after the mutation, and wherein verifying comprises recomputing the state hashes by replay and comparing them to the stored state hashes.

5. The method of claim 1, wherein the canonical receipts identify candidate memory records that were observed but not materialized, and the method further comprises, responsive to a query, returning a said candidate memory record together with a gate, a threshold, the policy version digest, and an evidence pointer that explain a non-materialization of the candidate memory record.

6. The method of claim 1, wherein the memory store includes a context-visible working tier and a full evidence tier, the working tier being a default retrieval surface read through an index, and wherein a canonical receipt records a demotion of a memory record from the context-visible working tier while retaining the memory record in the full evidence tier.

7. The method of claim 1, wherein the named memory-formation policy object is resolved from a layered precedence of at least two of a tenant policy, an agent policy, a job policy, a scope policy, a dream mode, and a request override, and wherein the policy version digest binds the resolved policy object to the layered precedence.

8. The method of claim 1, wherein applying the policy object comprises rejecting a candidate memory record when a resolved memory type of the candidate memory record is not in the allowed memory-type set, and recording in the canonical receipt a type-filter decision reason and the resolved memory type.

9. The method of claim 1, wherein applying the policy object comprises transforming a candidate memory record by redaction before materialization, and wherein the canonical receipt records a redaction digest before the transformation and a redaction digest after the transformation.

10. The method of claim 9, further comprising protecting a sensitive payload field of the canonical receipt with a destroyable content key and destroying the content key to render the sensitive payload field unrecoverable while retaining a receipt skeleton and a graph lineage sufficient to verify the tamper-evident sequence.

11. The method of claim 1, wherein applying the policy object comprises reinforcing an existing relationship of the memory store when a semantic similarity score between the candidate memory record and the existing relationship exceeds a policy threshold, and wherein the canonical receipt records the semantic similarity score, the policy threshold, and an identifier of the existing relationship.

12. The method of claim 1, wherein applying the policy object comprises superseding an active relationship of the memory store that shares a single-active truth key with the candidate memory record, and wherein the canonical receipt records an identifier of the superseded relationship and an identifier of a successor relationship.

13. The method of claim 1, wherein applying the policy object comprises blocking a candidate memory record that is a directive or a requirement extracted from an untrusted source pending an approval, and recording in the canonical receipt a trust-gate decision reason.

14. The method of claim 1, further comprising applying the named memory-formation policy object to a fixture corpus, verifying the linked canonical receipts and a graph state hash of a resulting run, and emitting a certificate comprising the policy version digest, a corpus digest, a Merkle root of the run, a plurality of certification metrics, and a pass/fail status.

15. A system comprising one or more processors and memory storing instructions that, when

executed, cause the system to perform the method of any of claims 1–14; and a non-transitory computer-readable medium storing instructions that, when executed, cause one or more processors to perform the method of any of claims 1–14.

Invention II — Counterfactual policy evaluation

16. A computer-implemented method for evaluating a memory-formation policy, comprising: retrieving, from a tamper-evident sequence of canonical receipts of a recorded memory-formation run, a candidate memory record having a candidate digest computed before application of a first named memory-formation policy object and an evidence digest of an episode from which the candidate memory record was extracted; resolving a second named memory-formation policy object different from the first; applying the second policy object to the candidate memory record to determine a counterfactual disposition; producing a counterfactual memory-store delta from a plurality of said counterfactual dispositions without mutating a production memory store; and emitting a report comparing the counterfactual memory-store delta to a production memory-store delta of the recorded run.

17. The method of claim 16, wherein the report identifies at least one of a candidate accepted by both policy objects, a candidate accepted only by the first policy object, a candidate accepted only by the second policy object, and a sensitive candidate that would be persisted under one policy object but not the other.

18. The method of claim 16, wherein applying the second policy object is performed without invoking a language model by reusing the candidate digest computed before application of the first policy object, whereby the counterfactual delta is derived deterministically from recorded receipts.

19. The method of claim 16, further comprising verifying the tamper-evident sequence of canonical receipts before applying the second policy object, and withholding the report when a receipt hash, a policy version digest, or an evidence digest does not match.

20. A system comprising one or more processors and memory storing instructions that, when executed, cause the system to perform the method of any of claims 16–19; and a non-transitory computer-readable medium storing instructions that, when executed, cause one or more processors to perform the method of any of claims 16–19.

ABSTRACT

An autonomous agent memory system records a canonical, tamper-evident receipt for each write-side memory decision. As candidate memories are extracted from an evidence episode and admitted, transformed, reinforced, superseded, demoted, gated, or rejected under a named memory-formation policy object, each receipt binds the decision to a digest of the immutable evidence, a digest of the resolved policy, a digest of the candidate, a decision reason, and the resulting memory-store pointers. The receipts are canonicalized, hash-linked, and committed into a per-run Merkle root, so that a memory-store state can be replayed and verified without invoking a language model, can be counterfactually regenerated under a different policy without mutating production memory, and can be queried for candidates that were observed but deliberately not persisted. Verification fails closed on any missing artifact or hash mismatch, a context-visible working tier is read through an index while a full evidence tier is preserved, and a named policy can be certified over a fixture corpus by a receipt-backed certificate.

FIG. 1

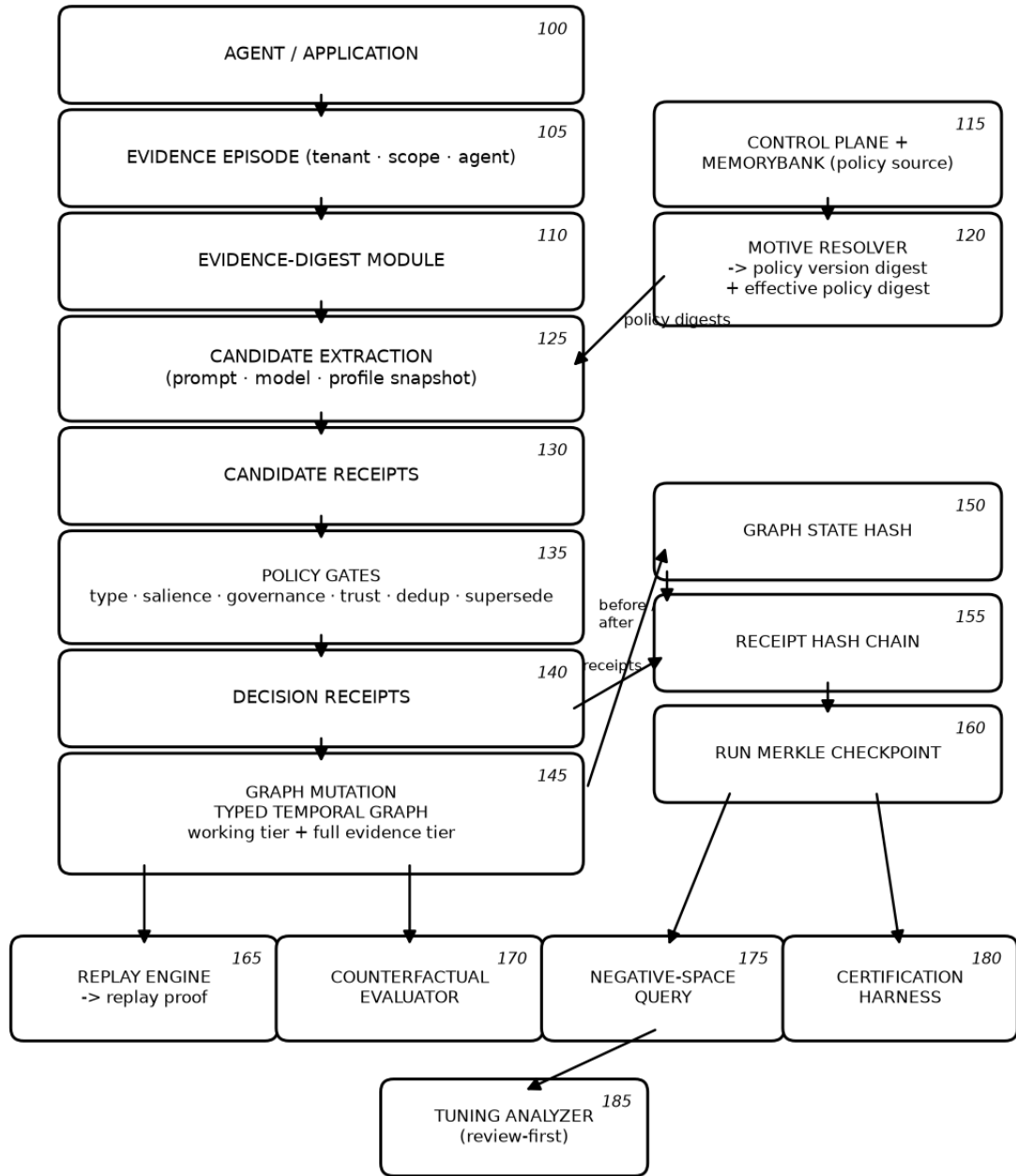


FIG. 1 — Layered architecture of the receipt-backed memory-formation system.

FIG. 2

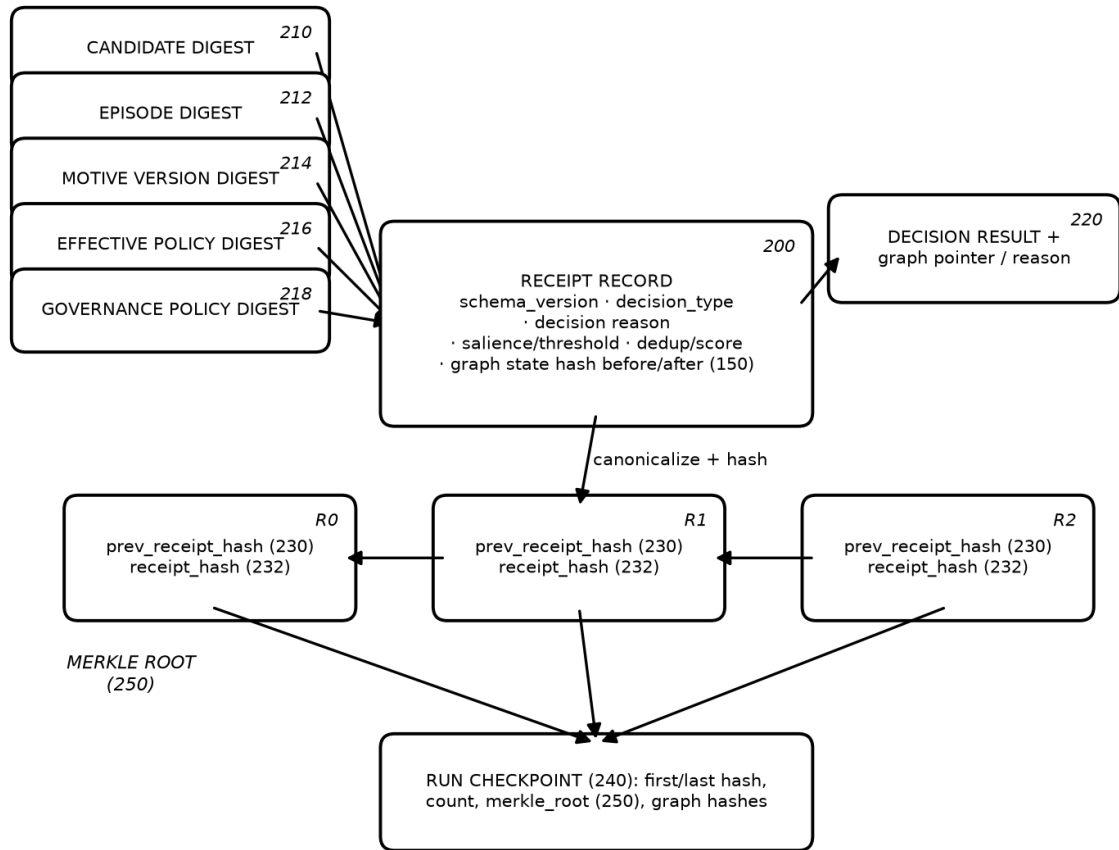


FIG. 2 — Receipt record, receipt hash chain, and per-run Merkle checkpoint.

FIG. 3

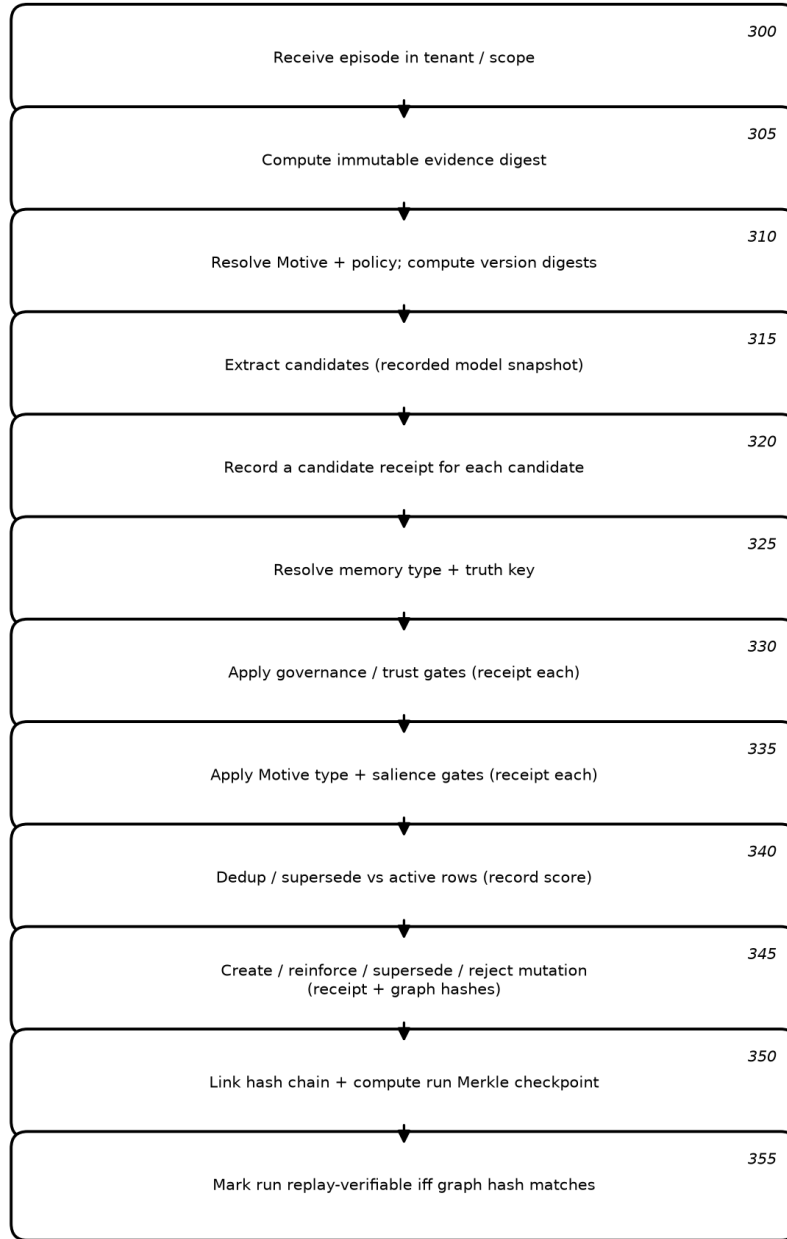


FIG. 3 — Receipt-backed memory-formation method.

FIG. 4

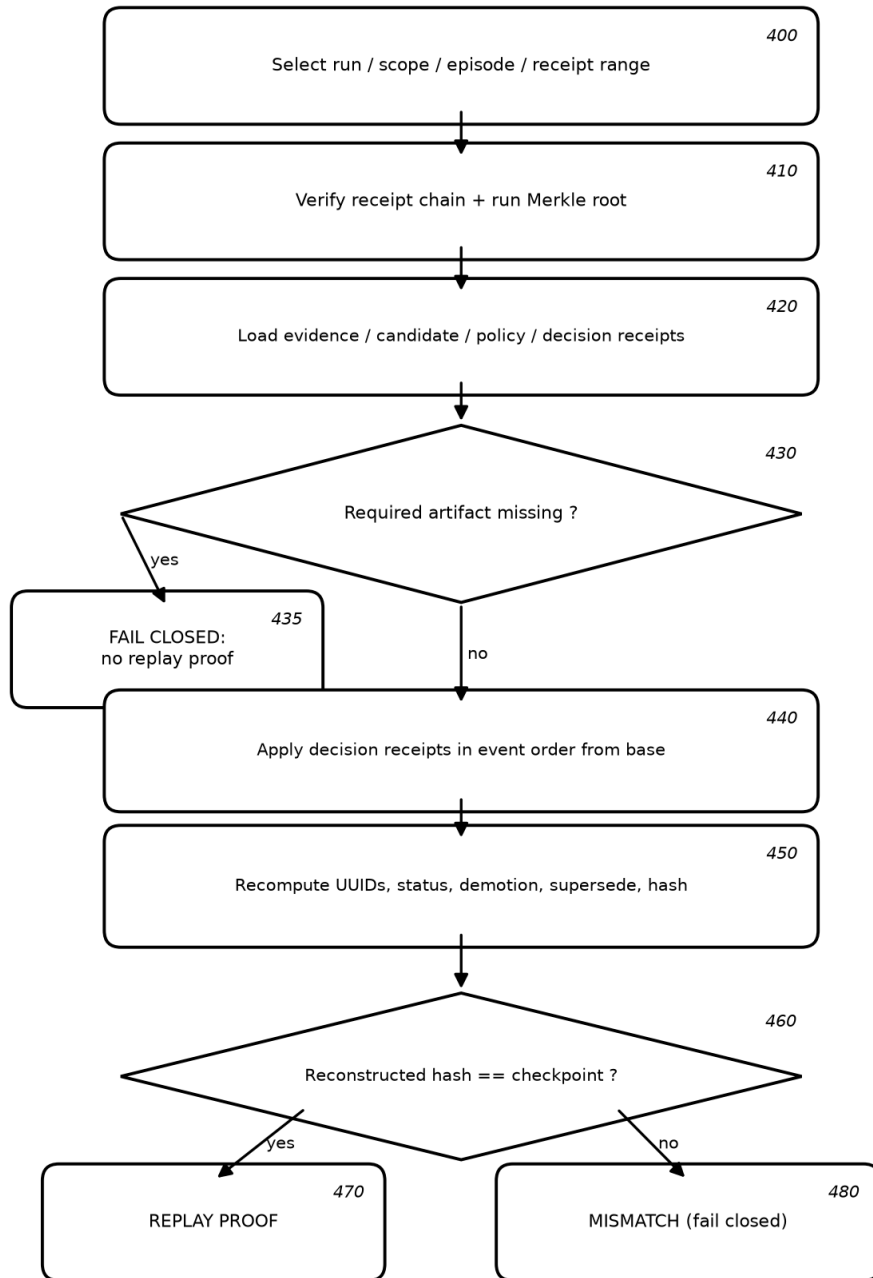


FIG. 4 — Byte-replay method with fail-closed verification gate.

FIG. 5

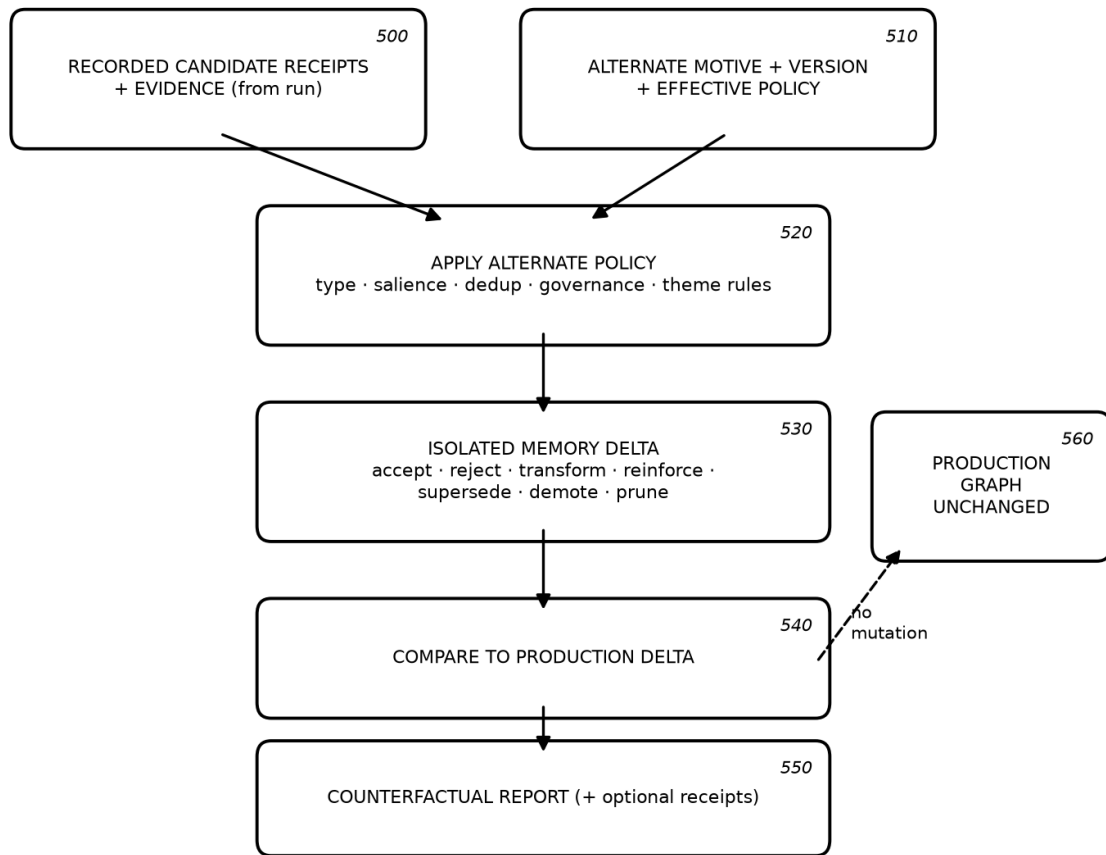


FIG. 5 — Counterfactual policy evaluation without production mutation.

FIG. 6

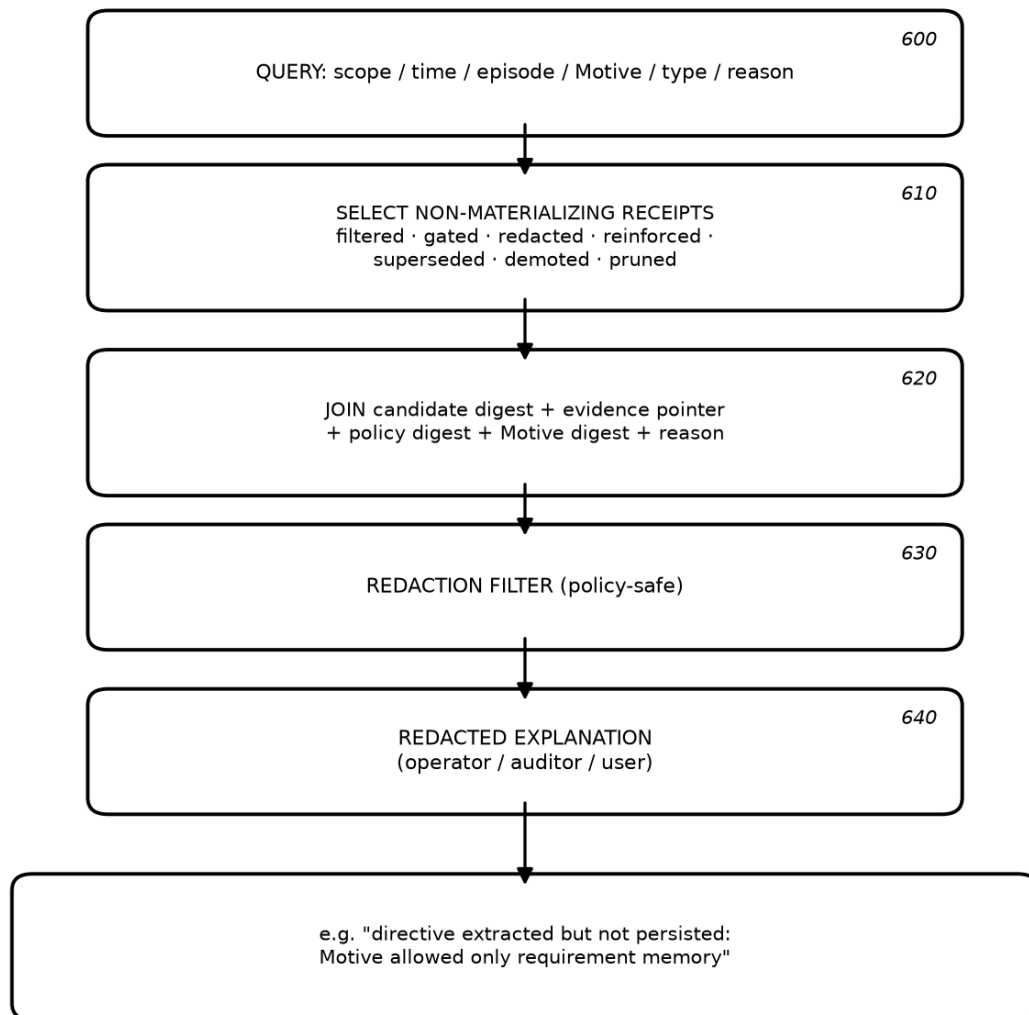


FIG. 6 — Negative-space memory query.

FIG. 7

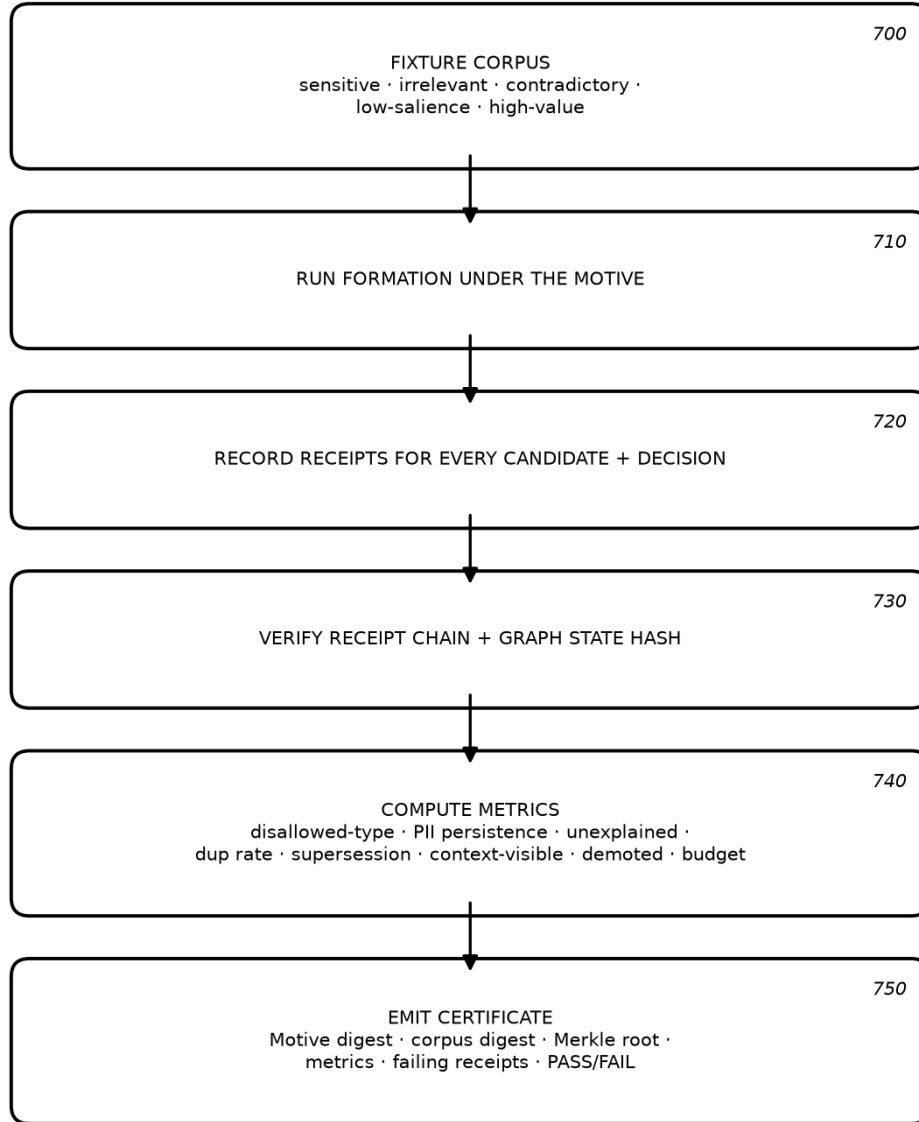


FIG. 7 — Policy certification harness over a fixture corpus.